

同济大学计算机系

计算机组成原理课程实验报告



学 号 2351520

姓 名 毛星博

专 业 计算机科学与技术

授课老师 陈永生

32 位无符号除法器

一、实验内容

无符号除法器 无符号除法器功能为将两个 32 位无符号数相除，得到一个 32 位商和 32 位余数。本实验分别实现 32 位有符号和无符号除法器，结果为 32 位商 quotient 和 32 位余数 remainder，分别存放在 CPU 的专用寄存器 LO 和 HI 中。除法器时钟信号下降沿时检查 start 信号，有效时开始执行，执行除法指令时，busy 标志位置 1。在执行除法指令时，任何情况下不产生算术异常，当除数为 0 时，运算结果未知，对除法器除数为 0 和溢出情况的发生通过汇编指令中其他指令进行检查和处理。

二、硬件逻辑图：无

三、模块建模

```
`timescale 1ns / 1ps

module DIVU(
    input [31:0] dividend,    // 被除数
    input [31:0] divisor,     // 除数
    input start,              // 启动除法运算
    input clock,
    input reset,
    output [31:0] q,          // 商
    output [31:0] r,          // 余数
    output reg busy           // 除法器忙标志位
);

    wire ready;
    reg [5:0] count;          // 计数器，用于 32 位运算需要 6 位
    reg [31:0] reg_q;         // 商寄存器
    reg [31:0] reg_r;         // 余数寄存器
    reg [31:0] reg_b;         // 除数寄存器
    reg busy2, r_sign;        // busy2 用于产生 ready 信号，r_sign 表示余数符号

    // ready 信号生成
    assign ready = ~busy & busy2;

    // 加减法器，根据 r_sign 选择加法或减法
    wire [32:0] sub_add = r_sign ?
        ({reg_r, reg_q[31]} + {1'b0, reg_b}) :
        ({reg_r, reg_q[31]} - {1'b0, reg_b});
```

```

// 最终余数计算
assign r = r_sign ? reg_r + reg_b : reg_r;
// 最终商输出
assign q = reg_q;

// 主状态控制
always @(posedge clock or posedge reset) begin
    if (reset == 1) begin
        // 复位状态
        count <= 6'b0;
        busy <= 0;
        busy2 <= 0;
    end else begin
        busy2 <= busy;
        if (start) begin
            // 开始除法运算，初始化各寄存器
            reg_r <= 32'b0;
            r_sign <= 0;
            reg_q <= dividend;
            reg_b <= divisor;
            count <= 6'b0;
            busy <= 1'b1;
        end else if (busy) begin
            // 循环操作
            reg_r <= sub_add[31:0]; // 更新部分余数
            r_sign <= sub_add[32]; // 更新符号位，决定下次是加还是减
            reg_q <= {reg_q[30:0], ~sub_add[32]}; // 更新商
            count <= count + 6'b1; // 计数器加 1
            if (count == 6'h1F) // 32 位运算完成
                busy <= 0; // 清除忙标志
        end
    end
end
end

endmodule

```

四、测试模块建模

```

`timescale 1ns / 1ps

module DIVU_tb;
    // 定义信号
    reg [31:0] dividend;

```

```

reg [31:0] divisor;
reg start;
reg clock;
reg reset;
wire [31:0] q;
wire [31:0] r;
wire busy;

// 实例化被测试模块
DIVU uut (
    .dividend(dividend),
    .divisor(divisor),
    .start(start),
    .clock(clock),
    .reset(reset),
    .q(q),
    .r(r),
    .busy(busy)
);

// 时钟生成
initial begin
    clock = 0;
    forever #5 clock = ~clock;
end

// 测试激励
initial begin
    // 初始化信号
    dividend = 0;
    divisor = 0;
    start = 0;
    reset = 1;

    // 等待 100ns 后开始测试
    #100;
    reset = 0;

    // 测试用例 1: 基本除法运算 (20/3)
    dividend = 32'd20;
    divisor = 32'd3;
    start = 1;
    #10 start = 0;
    wait(!busy);

```

```

if(q === 32'd6 && r === 32'd2) begin
    // Test Case 1 Passed
end else begin
    // Test Case 1 Failed
end
#20;

// 测试用例 2: 除数为 1
dividend = 32'd100;
divisor = 32'd1;
start = 1;
#10 start = 0;
wait(!busy);
if(q === 32'd100 && r === 32'd0) begin
    // Test Case 2 Passed
end else begin
    // Test Case 2 Failed
end
#20;

// 测试用例 3: 被除数为 0
dividend = 32'd0;
divisor = 32'd5;
start = 1;
#10 start = 0;
wait(!busy);
if(q === 32'd0 && r === 32'd0) begin
    // Test Case 3 Passed
end else begin
    // Test Case 3 Failed
end
#20;

// 测试用例 4: 大数除法
dividend = 32'hFFFFFFFF;
divisor = 32'h0000FFFF;
start = 1;
#10 start = 0;
wait(!busy);
if(q === 32'h00010001 && r === 32'h00000000) begin
    // Test Case 4 Passed
end else begin
    // Test Case 4 Failed
end

```

```

#20;

// 测试复位功能
dividend = 32'd100;
divisor = 32'd2;
start = 1;
#10;
reset = 1;
#10;
if(q === 32'd0 && r === 32'd0) begin
    // Reset Test Passed
end else begin
    // Reset Test Failed
end

// 结束仿真
#100 $finish;
end

// 监控 busy 信号变化
always @(busy) begin
    if(busy) begin
        // Division Started
    end else begin
        // Division Completed
    end
end
end

endmodule

```

五、实验结果

Modelsim 仿真图

